

Código 1

```
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    static final String ALUNO = "Lucas Gabriel Godoy de Oliveira";
    static final String RA = "24214463-2";

    static final String ENTRADA = "resultado = (valor1 - valor2) + valor3 * 75;";

    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();

    static {
        SIMBOLOS.put('=', "ATRIBUICAO");
        SIMBOLOS.put('+', "SOMA");
        SIMBOLOS.put('-', "SUBTRACAO");
        SIMBOLOS.put('*', "MULTIPLICACAO");
        SIMBOLOS.put('/', "DIVISAO");
        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
        SIMBOLOS.put('(', "ABRE_PARENTESSES");
        SIMBOLOS.put(')', "FECHA_PARENTESSES");
    }
}
```

ALUNO: Lucas Gabriel Godoy de Oliveira | RA: 24214463-2
ENTRADA: resultado = (valor1 - valor2) + valor3 * 75;

LEXEMA	TOKEN
resultado	IDENTIFICADOR
=	ATRIBUICAO
(	ABRE_PARENTESSES
valor1	IDENTIFICADOR
-	SUBTRACAO
valor2	IDENTIFICADOR
)	FECHA_PARENTESSES
+	SOMA
valor3	IDENTIFICADOR
*	MULTIPLICACAO
75	INTEIRO
;	PONTO_E_VIRGULA

Código 2

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

public class Main {
    static final String ALUNO = "Lucas Gabriel Godoy de Oliveira";
    static final String RA = "24214463-2";

    static final String IDENTIFICADOR_PERSONALIZADO = "LUCAS";

    static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>() {
        {
            Arrays.asList("int", "if", "else", "while", "return")
        }
    };

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
        executarTeste("TESTE A - entrada compacta",
            "int total=valor10;");
        executarTeste("TESTE B - espaços e comentário",
            "int total = valor + 10; // este comentario sera ignorado");
        executarTeste("TESTE C - reservadas e identificadores",
            "int inteiro = 0; while (inteiro < intValue) inteiro = inteiro + 1;");
        executarTeste("TESTE C2 - identificador personalizado",
            "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
        executarTeste("TESTE D - caractere inválido",
            "int valor# = 1;");
    }

    static void executarTeste(String titulo, String entrada) {
        System.out.println();
        System.out.println("=== " + titulo + " ===");
        System.out.println("ENTRADA: " + entrada);
        System.out.printf("%-22s %-22s %s\n", "LEXEMA", "CLASSE", "TOKEN");
    }
}
```

LEXEMA	CLASSE	TOKEN
int	LETRA	PALAVRA_RESERVADA
LUCAS	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===
ENTRADA: int LUCAS = 10;
LEXEMA CLASSE TOKEN

LEXEMA	CLASSE	TOKEN
int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

Código 3

```
public class Main {
    static final String ALUNO = "Lucas Gabriel Godoy de Oliveira";
    static final String RA = "24214463-2";

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);

        int total = 10 + 20; // CORRIGIR 1: acrescente o ponto e virgula

        if (total > 20) { // CORRIGIR 2: feche corretamente o parenteses
            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
            System.out.println("Resultado: " + total);
        }
    }
}
```

ALUNO: Lucas Gabriel Godoy de Oliveira | RA: 24214463-2
Programa corrigido!
Resultado: 30

Código 4

```
public class Main {
    static final String ALUNO = "Lucas Gabriel Godoy de Oliveira";
    static final String RA = "24214463-2";

    static final String ENTRADA = "(3 + 3) * 4";
    static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
        System.out.println("ENTRADA: " + ENTRADA);
        System.out.println();

        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
            demonstrarRecursoEsquerda(0);
            System.out.println();
        }

        try {
            Parser parser = new Parser(ENTRADA);
            int resultado = parser.analisar();
            System.out.println();
            System.out.println("ENTRADA ACEITA");
            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
        } catch (ErroSintatico erro) {
            System.out.println();
            System.out.println("ERRO SINTATICO: " + erro.getMessage());
        }
    }

    static void demonstrarRecursoEsquerda(int nivel) {
        System.out.println(recuo(nivel)
            + "expr -> expr + term (chamada " + (nivel + 1) + ")");
    }
}
```

```
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead: +
SAINDO de <factor> | lookahead = +
SAINDO de <term> | lookahead = +
Encontrado operador +
Novo lookahead: 3
ENTRANDO em <term> | lookahead = 3
ENTRANDO em <factor> | lookahead = 3
Numero reconhecido: 3
Novo lookahead: )
SAINDO de <factor> | lookahead = )
SAINDO de <term> | lookahead = )
SAINDO de <expr> | lookahead = )
Novo lookahead: *
Fechando parenteses
SAINDO de <factor> | lookahead = *
Encontrado operador *
Novo lookahead: 4
ENTRANDO em <factor> | lookahead = 4
Numero reconhecido: 4
Novo lookahead: $FIM$
SAINDO de <factor> | lookahead = $FIM$
SAINDO de <term> | lookahead = $FIM$
SAINDO de <expr> | lookahead = $FIM$

ENTRADA ACEITA
RESULTADO DA EXPRESSAO: 24
```

Código 5

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Main {
    static final String ALUNO = "Lucas Gabriel Godoy de Olivera";
    static final String RA = "24214463-2";

    static final String ENTRADA_VALIDA = "id*id*id";
    static final String ENTRADA_INVALIDA = "id*id";

    // Na primeira execução, use ENTRADA_VALIDA. Depois troque por ENTRADA_INVALIDA.
    static final String ENTRADA = ENTRADA_INVALIDA;

    static final Map<Integer, Map<String, String>> ACTION = new HashMap<>();
    static final Map<Integer, Map<String, Integer>> GOTO = new HashMap<>();
    static final Map<Integer, Regra> REGRAS = new HashMap<>();

    static {
        // Gramática:
        // 1) E -> E + T      2) E -> T
        // 3) T -> T * F      4) T -> F
        // 5) F -> ( F )      6) F -> id
    }
}
```

```
ALUNO: Lucas Gabriel Godoy de Olivera | RA: 24214463-2
ENTRADA: id*id*id

PILHA          ENTRADA
ACAO
-----
$ 0            id*id$
SHIFT id -> estado 5
$ 0 id 5      +*id$
REDUCE F -> id
$ 0 F 3       +*id$
REDUCE T -> F
$ 0 T 2       +*id$
REDUCE E -> T
$ 0 E 1       +*id$
SHIFT + -> estado 6
$ 0 E 1 + 6   *id$
ERROR: nenhuma ação para estado 6 e símbolo *
```